CMPSC 465, Fall 2025, HW 4

Mohammad Abu Shariha
mma7020@psu.edu

Problem 1(a)

A Depth-First Search (DFS) is performed on the graph from the problem sheet. The main DFS loop considers starting nodes alphabetically. For any given node, its neighbors are also explored in alphabetical order.

DFS Timestamps

The traversal yields the following discovery (`pre`) and finishing (`post`) times for each vertex.

Vertex	Pre Number	Post Number
A	1	12
B	2	7
C	13	14
D	15	16
E	4	5
F	3	6
G	8	9
H	10	11

Edge Classification

Each edge in the graph is classified based on the DFS traversal.

Edge	Type
(A, B)	Tree Edge
(A, G)	Tree Edge
(A, H)	Tree Edge
(B, F)	Tree Edge
(F, E)	Tree Edge
(F, B)	**Back Edge**
(C, B)	Cross Edge
(C, F)	Cross Edge
(D, C)	Cross Edge
(D, E)	Cross Edge
(G, F)	Cross Edge
(H, G)	Cross Edge

There were no **Forward Edges** produced in this specific traversal.

Problem 1(b)

Statement: If $\{u, v\}$ is an edge in an undirected graph, and during depth-first search $post(u) < post(v)$, then v is an ancestor of u in the DFS tree.

This statement is **true**.

Proof

In a DFS of an undirected graph, there are only two types of edges: tree edges and back edges. There are no forward or cross edges. The discovery and finishing times of any two vertices have a parenthesis structure: their time intervals $[pre(x), post(x)]$ and $[pre(y), post(y)]$ are either disjoint or one is nested inside the other.

We consider the two possible discovery orders for u and v:

1. **Case 1: u is discovered before v (i.e., $pre(u) < pre(v)$).** Since $\{u, v\}$ is an edge, the traversal will discover v while exploring from u (if v is unvisited). This makes v a descendant of u in the DFS tree. In this scenario, the recursive call $DFS(v)$ is made and completes entirely within the scope of the $DFS(u)$ call. This implies the nesting of time intervals: $pre(u) < pre(v) < post(v) < post(u)$. The result $post(v) < post(u)$ contradicts the given condition that $post(u) < post(v)$. Therefore, this case is not possible.

2. **Case 2: v is discovered before u (i.e., $pre(v) < pre(u)$).** Since $\{u, v\}$ is an edge, u must be discovered during the exploration of v or one of its descendants. This makes u a descendant of v. The $DFS(u)$ call must complete before the $DFS(v)$ call can complete. This implies the time interval nesting: $pre(v) < pre(u) < post(u) < post(v)$. The result $post(u) < post(v)$ is consistent with the given condition.

Since only the second case is consistent with the premise, it must be that v is discovered before u and u is a descendant of v. Thus, v is an ancestor of u.

Problem 2(a)

A **source** is a vertex with an in-degree of 0. A **sink** is a vertex with an out-degree of 0. For the given directed graph:

- **Sources**: A, B
- **Sinks**: G, I, J

Problem 2(b)

We run the DFS-based topological sort algorithm. We start a DFS, and as each vertex finishes, we prepend it to a list. We process unvisited starting nodes in alphabetical order (A, then B), and for any node, we visit its neighbors in alphabetical order.

The algorithm produces the following topological ordering:

B, E, A, D, H, I, C, F, J, G

Problem 2(c)

Statement: Prove or disprove that the ordering $S = (A, C, B, D, F, J, E, H, I, G)$ can be obtained by the algorithm.

This statement is **true**, but it depends on the specific topological sort algorithm used and the choices made during its execution.

1. **Is S a valid topological ordering?**
   Yes. A check of all directed edges $u \to v$ in the graph confirms that u always appears before v in the sequence S.

2. **Can a deterministic algorithm like Kahn's (with alphabetical tie-breaks) produce S?**
   No. Kahn's algorithm begins with a set of all sources, which is $\{A, B\}$. An alphabetical tie-break rule would force the algorithm to select A first. After processing A, vertex C becomes a source. The set of sources is now $\{B, C\}$. The alphabetical tie-break would then force the selection of B. Thus, this algorithm must produce a sequence beginning with $A, B, \ldots$. The sequence S begins with $A, C, \ldots$ and is therefore unobtainable with this method.

3. **Can the DFS-based algorithm produce S?**
   Yes. The output of the DFS-based algorithm depends on the choices made during traversal (the order of starting nodes for the DFS forest and the order of visiting neighbors). To produce the sequence S, the vertices must finish in the reverse order: $(G, I, H, E, J, F, D, B, C, A)$. A valid DFS traversal can be constructed to produce this exact finishing order by altering the choice of starting nodes (e.g., starting the main loop at B instead of A) and the order in which neighbors are visited.

   Since a valid DFS traversal exists that produces this order, the ordering S **can be obtained** by the DFS-based algorithm.

Problem 3(a)

Algorithm

To determine if a feasible route from city s to city t exists given a water bottle capacity of L:

1. **Construct Subgraph:** Create a new graph $G_L = (V, E')$ that includes all cities V, but only includes edges $e \in E$ from the original graph if their length $\ell_e \leq L$.
2. **Check Reachability:** Run a Breadth-First Search (BFS) or Depth-First Search (DFS) on G_L starting from city s.
3. **Conclusion:** If the search algorithm visits city t, a feasible route exists. Otherwise, no such route exists.

Correctness

The subgraph G_L represents all possible movements allowed by the water bottle's capacity. A path exists from s to t in G_L if and only if there is a sequence of roads, each of length at most L, connecting s to t. BFS/DFS is a standard and correct method for determining reachability in a graph.

Runtime Analysis

Constructing G_L requires iterating through all edges, taking $O(|E|)$ time. Running BFS or DFS on G_L takes $O(|V| + |E'|)$. Since $|E'| \leq |E|$, the total time is $O(|E|) + O(|V| + |E|) = O(|V| + |E|)$, which is linear.

Problem 3(b)

Algorithm

The problem of finding the minimum required bottle capacity is equivalent to finding a "bottleneck shortest path"—a path from s to t that minimizes the maximum edge weight along the path. This can be solved with a modification of Dijkstra's algorithm.

1. **Initialization:**
   - For each city v, create an attribute $min_capacity[v]$, initialized to ∞. Set $min_capacity[s] = 0$.
   - Use a min-priority queue, PQ, that stores tuples of $(capacity, city)$ and is prioritized by the smallest capacity. Add $(0, s)$ to PQ.
2. **Main Loop:**
   - While PQ is not empty, extract the city u with the smallest priority (minimum capacity c) from PQ.
   - If u is the destination city t, return c as the answer.
   - For each neighbor v of u connected by a road e with length ℓ_e:
     - The capacity needed to travel to v via the current path is $new_bottleneck = \max(c, \ell_e)$.
     - If $new_bottleneck < min_capacity[v]$, update $min_capacity[v] = new_bottleneck$ and add/update its entry in the PQ.

Correctness

The algorithm is a greedy approach that always explores from the city that is reachable with the smallest bottleneck found so far. The structure is identical to Dijkstra's, and the same proof of correctness applies: when a city u is extracted from the priority queue, its minimum capacity path has been finalized.

Runtime Analysis

The algorithm's complexity is identical to Dijkstra's algorithm. Using a binary heap for the priority queue, the runtime is $O((|V| + |E|) \log |V|)$.

Problem 4

Algorithm

To determine if a specific edge $e = \{u, v\}$ is part of any cycle in an undirected graph G, we can check if there's an alternative path between u and v.

1. **Remove Edge:** Temporarily remove the edge $e = \{u, v\}$ from the graph G, resulting in a new graph G'.
2. **Check for Path:** In G', perform a graph traversal (like BFS or DFS) starting from vertex u.
3. **Conclusion:** Check if vertex v was visited during the traversal.
    - If v is reachable from u in G', it means there is an alternative path between them. This path, combined with the original edge e, forms a cycle.
    - If v is not reachable from u in G', then no alternative path exists, and the edge e cannot be part of a cycle.

Correctness

A cycle containing edge $\{u, v\}$ is, by definition, the edge $\{u, v\}$ itself plus a path from v to u that does not use the edge $\{u, v\}$. The algorithm correctly searches for the existence of this alternate path.

Runtime Analysis

The runtime is dominated by the single graph traversal (BFS or DFS), which is $O(|V| + |E|)$.

Problem 5(a)

Algorithm

It is possible to complete all courses if and only if the prerequisite graph G contains no directed cycles, i.e., it is a Directed Acyclic Graph (DAG). We can detect cycles using DFS.

1. Maintain three sets (or colors) for each vertex: `UNVISITED`, `VISITING` (currently in the recursion stack), and `VISITED`.
2. Iterate through each vertex u in the graph. If u is `UNVISITED`, start a recursive DFS from u.
3. In the recursive function $DFS(u)$:
   - Move u from `UNVISITED` to `VISITING`.
   - For each neighbor v of u:
     - If v is in the `VISITING` set, a back edge has been found, which indicates a cycle. The algorithm can terminate and report that it's impossible to complete all courses.
     - If v is `UNVISITED`, recursively call $DFS(v)$.
   - Move u from `VISITING` to `VISITED`.
4. If the DFS completes for all vertices without finding a cycle, the graph is a DAG.

Runtime Analysis

This is a standard DFS traversal where each vertex and edge is visited once. The runtime is $O(|V| + |E|)$.

Problem 5(b)

Algorithm

A valid sequence to take all courses is a topological sort of the prerequisite graph.

1. First, run the algorithm from part (a) to ensure the graph is a DAG. If it's not, report that no valid sequence exists.
2. If the graph is a DAG, perform a topological sort. A standard algorithm is:
   - Run DFS on the graph.
   - As each vertex u finishes (i.e., after the recursive calls for all its neighbors have returned), add u to the front of a linked list.
3. The final linked list represents a valid sequence of courses.

Runtime Analysis

This algorithm is based on a single DFS traversal, so its runtime is $O(|V| + |E|)$.

Problem 5(c)

Algorithm

The minimum number of semesters needed to complete the curriculum is determined by the length (number of vertices) of the **longest path** in the prerequisite DAG.

1. **Topological Sort:** Compute a topological ordering of the courses, let's say $v_1, v_2, \ldots, v_n$.
2. **Initialize Semesters:** Create an array $semesters[v]$ for all courses v, and initialize all values to 1. $semesters[v]$ will store the earliest semester in which course v can be taken.
3. **Calculate Longest Paths:** Iterate through the courses u in the computed topological order:
   - For each course v that has u as a direct prerequisite (i.e., for each edge $u \to v$):
   - Update $semesters[v] = \max(semesters[v], semesters[u] + 1)$.
4. **Find Maximum:** The minimum number of semesters required is the maximum value in the $semesters$ array, i.e., $\max_{v \in V}(semesters[v])$.

Correctness

By processing courses in topological order, we ensure that when we compute the earliest semester for a course v, we have already computed the values for all of its prerequisites. The update rule guarantees that v is scheduled in a semester strictly after all of its prerequisites. The final result is dictated by the longest chain of dependencies, which is precisely what the algorithm calculates.

Runtime Analysis

The runtime is dominated by the initial topological sort, making the total runtime $O(|V| + |E|)$.